| | |
|---|---|
| **CS3230 Design and Analysis of Algorithms** | October 26, 2019 |

Applications of Flows and Cuts, NP-Completeness

*Name: Divesh Aggarwal*

## Problem 9-1 (Passes Through)

Given an undirected graph $G = (V, E)$, with three vertices $u$, $v$, and $w$, describe and analyze an algorithm to determine whether there is a path from $u$ to $w$ that passes through $v$. (**Hint**: If $G$ were a directed graph, this problem would be NP-hard!)

**Solution:**

We reduce the Passes Through problem to the Vertex-Disjoint Paths problem. On input the undirected graph $G$, we construct the directed graph $H$ as follows. First, for every edge $xy$ in $G$, we add directed edges $x \to y$ and $y \to x$. Next, we add a new vertex $z$, and two new edges $u \to z$ and $w \to z$.

We then compute the maximum number of vertex-disjoint paths from $v$ to $z$ in $H$, using the algorithm solving the Vertex-Disjoint Paths problem in $O(VE)$ described in the lecture. If the number we get is two, we return TRUE; and we return FALSE otherwise.

We show that our reduction is correct. Suppose $G$ has a path $P$ from $u$ to $w$ that passes through $v$. Let $P_{uv}$ be a path from $u$ to $v$ and $P_{vw}$ be a path from $v$ to $w$. Since $G$ is undirected, we also have a path $P_{vu}$ from $v$ to $u$ in $G$. Notice that by construction, $P_{vu}$ and $P_{vw}$ are vertex-disjoint paths in $H$ as well. Since we connected the ends of both paths $P_{vu}$ and $P_{vw}$ to the new vertex $z$, there are two vertex-disjoint paths from $v$ to $z$ in $H$.

Now suppose $H$ contains two vertex-disjoint paths $P_1$ and $P_2$ from $v$ to $z$. We remove the last edge ($u \to z$ and $w \to z$) from each of the paths; and let $P_{vu}$ and $P_{vw}$ be two resulting paths. Notice that by construction, $P_{vu}$ and $P_{vw}$ are also paths in $G$ that only share a common endpoint $v$. Thus, $P_{uv}||P_{vw}$ is a path from $u$ to $w$ that passes through $v$.

□

## Problem 9-2 (Cycle Cover)

A cycle cover of a given directed graph $G = (V, E)$ is a set of vertex-disjoint cycles that cover every vertex in $G$. Describe and analyze an efficient algorithm to find a cycle cover for a given graph, or correctly report that no cycle cover exists. (**Hint**: Use bipartite matching!)

**Solution:**

Given the directed graph $G$, we construct the bipartite graph $H$ as follows. For each vertex $v$ in $G$, we add two vertices $v_{in}$ and $v_{out}$ in $H$. For each edge $u \to v$ in $G$, we add an edge $u_{out}v_{in}$ in $H$.

We claim that $G$ has a cycle cover iff $H$ has a maximum matching. Suppose $G$ has a cycle cover $C$. Let $M$ be the collection of edges $u_{out}v_{in}$ in $H$ for all directed edges $u \to v$ in $C$. Since each vertex in $G$ is incident to exactly one incoming edge and exactly one outgoing edge in the cycle

cover $C$, by our construction each vertex in $H$ is incident to exactly one edge in $M$. Thus, $M$ is a maximum matching in $H$.

Now suppose $H$ has a maximum matching $M$. Let $C$ be the collection of directed edges $u \to v$ in $G$ for all edges $u_{out}v_{in}$ in $M$. Since each vertex in $H$ is incident to exactly one edge in the maximum matching $M$, by our construction each vertex in $G$ is incident to exactly one incoming edge and exactly one outgoing edge in $C$. Thus, $C$ is a cycle cover in $G$.

Also, since we can compute a maximum matching or report none exists in $H$ in $O(VE)$ time using the algorithm described in lecture, and we can construct $H$ from $G$ and $C$ from $M$ in $O(V)$ time, the proof of claim above gives us an algorithm to find a cycle cover or report none exists in in $O(VE)$ time.

□

## Problem 9-3 (Hardness of MAX-2SAT and MAX-CUT)

(a) We want to analyze a reduction from 3SAT to the following language:

$$\mathsf{MAX\text{-}2SAT} = \{(\phi', k) \mid \phi' \text{ is a 2-CNF formula, and there is an assignment that satisfies at least } k \text{ clauses}\}.$$

Our reduction is the following: Given a 3SAT instance $\phi$, we will output a MAX-2SAT instance $(\phi', k)$, where $\phi'$ is a 2-CNF formula. To construct $\phi'$, do the following: for each clause $(x \vee y \vee z)$ in $\phi$, add the following 10 clauses to $\phi'$ (where $w$ is a fresh variable for each clause):

$$(x), (y), (z), (\neg x \vee \neg y), (\neg y \vee \neg z), (\neg x \vee \neg z), (w), (x \vee \neg w), (y \vee \neg w), (z \vee \neg w)$$

Find a value of $k$ such that $(\phi', k) \in \mathsf{MAX\text{-}2SAT}$ if and only if $\phi \in \mathsf{3SAT}$. Prove the correctness of the reduction.

**Solution:** Consider a certain clause $(x \vee y \vee z)$ in $\phi$.

If one of $x, y, z$ is satisfied (wlog $x$), then setting $w = \mathsf{false}$, we get $(x), (\neg x \vee \neg y), (\neg y \vee \neg z), (\neg x \vee \neg z), (x \vee \neg w), (y \vee \neg w), (z \vee \neg w)$ are satisfied.

If two of $x, y, z$ are satisfied (wlog $x, y$), then setting $w = \mathsf{false}$, we get $(x), (y), (\neg y \vee \neg z), (\neg x \vee \neg z), (x \vee \neg w), (y \vee \neg w), (z \vee \neg w)$ are satisfied.

If all of $x, y, z$ are satisfied, then setting $w = \mathsf{true}$, we get $(x), (y), (z), (w), (x \vee \neg w), (y \vee \neg w), (z \vee \neg w)$ are satisfied.

Now if none of $x, y, z$ are satisfied, then setting $w = \mathsf{true}$, we only get $(\neg x \vee \neg y), (\neg y \vee \neg z), (\neg x \vee \neg z), (w)$ are satisfied; and setting $w = \mathsf{false}$, we only get $(\neg x \vee \neg y), (\neg y \vee \neg z), (\neg x \vee \neg z), (x \vee \neg w), (y \vee \neg w), (z \vee \neg w)$ are satisfied.

Therefore, if $(x \vee y \vee z) = 1$ then 7 clauses of $\phi'$ are satisfiable, and if $(x \vee y \vee z) = 0$ then at most 6 clauses of $\phi'$ are satisfiable. Thus, we set $k = 7m$. □

Now consider the following language:

$$\mathsf{MAX\text{-}CUT} = \{(G, c) \mid G \text{ is a multigraph with a cut of size at least } c\}.$$

A *cut* in a graph is a partition of its vertices into two parts. The size of the cut is the number of edges which "cross" the cut (whose endpoints are in opposite parts). A multigraph means we allow duplicate edges.

We now analyze a reduction from MAX-2SAT to MAX-CUT. Given an instance $(\phi, k)$ of MAX-2SAT, let $n$ be the number of variables occuring in $\phi$, and $m$ the number of clauses. Consider the following graph: $G_\phi$ is a graph with a vertices labeled $x_i$ and $\neg x_i$ for each variable $x$ occuring in $\phi$, and two special vertices labeled $T$ and $F$. We add $5m$ edges between $T$ and $F$, and $5m$ edges between each pair $(x_i, \neg x_i)$ — see Figure 1. Then, for each clause $(x \vee y) \in \phi$, where $x$ and $y$ are literals, we add the following 7 edges (see clause subgraph in Figure 2): $(x, y)$, $(T, x)$, $(T, y)$, and two copies of the edges $(x, F)$ and $(y, F)$.

(b) Show that in the largest cut in $G_\phi$, $T$ and $F$ must be in opposite parts.

**Solution:** If $T$ and $F$ are in the same part, separating them can only increase the size of the cut by at least $5m - 4m = m$. □

(c) Show that in the largest cut in $G_\phi$, the vertices corresponding to $x$ and $\neg x$ must be in opposite parts.

**Solution:** If $x$ and $\neg x$ are in the same part, separating them can only increase the size of the cut by at least $5m - 4m = m$. □

(d) Argue that $(\phi, k) \in$ MAX-2SAT if and only if $(G_\phi,\ 5m + 5mn + 4k + 2(m - k)) \in$ MAX-CUT.

**Solution:** First we argue that if $(\phi, k) \in$ MAX-2SAT then $(G_\phi,\ 5m+5mn+4k+2(m-k)) \in$ MAX-CUT.

Suppose $\phi$ has an assignment satisfying $\geq k$ clauses. Create the following cut $(V_T, V_F)$: put $T$ in $V_T$ and put $F$ in $V_F$; for all $i \in [n]$, if $x_i$ is true then put $x_i$ in $V_T$, $\neg x_i$ in $V_F$; but if $x_i$ is false then put $x_i$ in $V_F$, $\neg x_i$ in $V_T$.

Since we separate $T$, $F$ and $x_i$, $\neg x_i \, \forall i$, this gives $5m + 5mn$ edges to the cut. Let $(x, y)$ be a satisfied clause. Either both were set true or just one of them, that clause gives 4 edges to the cut. Since there are $k$ satisfied clauses, they give $4k$ edges to the cut. Now let $(x, y)$ be an unsatisfied clause. Both were set false, so that clause gives 2 edges to the cut. Since there are $m - k$ satisfied clauses, they give $2(m - k)$ edges to the cut.

Next we argue that if $(G_\phi,\ 5m+5mn+4k+2(m-k)) \in$ MAX-CUT then $(\phi, k) \in$ MAX-2SAT.

Suppose $G_\phi$ has a cut of size $5m + 5mn + 4k + 2(m - k)$. By (b) and (c), $5m + 5mn$ edges of the cut account for separated $T$, $F$ and $x_i$, $\neg x_i \, \forall i$. We set the literals that belong to the same part as $T$ to true, and the ones with $F$ as false. The $4k$ edges account for either having both or one of the literal vertices in the same part with $T$ for each $k$ clause subgraph, which give $k$ satisfied clauses; and the $2(m - k))$ edges account for having both literal vertices in the same part with $F$ for each $m - k$ clause subgraph, which give $m - k$ unsatisfied clauses.

□

## Problem 9-4 (A polynomial time algorithm for 2SAT)

We want to show an efficient algorithm for 2SAT, which might be a little surprising since 3SAT is NP-hard. Given an instance $\phi$ of 2SAT, we first add a new literal $z$ to each one-literal clause. We then construct a directed graph $G = (V, E)$ with vertices $V$ labeled by the literals $x$ and $\overline{x}$ for each variable $x$ occuring in $\phi$. This graph has an edge $(u, v) \in E$ directed from vertex $u$ to vertex $v$ if the clause $(\overline{u} \vee v)$ is in $\phi$. If $(\overline{u} \vee v)$ is in $\phi$, so is $(v \vee \overline{u})$ because of commutativity of $\vee$. Thus, if $(u, v) \in E$, then $(\overline{v}, \overline{u}) \in E$ as well (See Figure 3). Note that $(u, v) \neq (\overline{v}, \overline{u})$ because this requires that $v = \overline{u}$, which is not allowed. Let $u \neq \overline{w}$. It follows that if there is a path from $u$ to $w$ in $G$, there is a distinct path from $\overline{w}$ to $\overline{u}$) obtained by reversing the directions of each edge on the path and replacing the literals by their complements.

To understand why these edges are chosen, note that if all clauses of $\phi$ are satisfied and $(\overline{u} \vee v)$ is in $\phi$, then $u = 1$ implies that $v = 1$. This implication relation, denoted $u \Rightarrow v$, is transitive. If there is a path $(u_1, u_2, \ldots, u_k)$ in $G$, then there are clauses $(\overline{u}_1 \vee u_2), (\overline{u}_2 \vee u_3), \ldots, (\overline{u}_{k-1} \vee u_k)$ in $\phi$. If all clauses are satisfied and if the literal $u_1 = 1$, then each unnegated literal on this path must have value 1.

(a) Show that if there exists a variable $x$ such that there is a path in $G$ from $x$ to $\overline{x}$ and from $\overline{x}$ to $x$, then $\phi$ is not satisfiable.

**Solution:** Suppose there is a variable $x$ such that such paths exists. This means that $x \Rightarrow \overline{x}$ and $\overline{x} \Rightarrow x$ which is a logical contradiction. This implies that $\phi$ is unsatisfiable. $\square$

(b) Hence show that if there if $\phi$ is not satisfiable, then there must be a literal $x$ such that there is a path from $x$ to $\overline{x}$, and from $\overline{x}$ to $x$. Conclude that 2SAT can be solved in polynomial time.

**Solution:** Suppose there isn't such a literal. Then, we can get a satisfying assignment as follows.

Recursively do the following.

1. Pick an arbitrary literal $\ell$ that hasn't been assigned a value and for which there is no path from $\ell$ to $\overline{\ell}$. Assign $\ell = 1$.
2. Then use DFS to visit every vertex reachable from $\ell$ and assign 1 to all the corresponding literals.

First we show that we didn't assign a value to a variable that was already assigned a different value in a previous step. Suppose we assigned $\ell_1 = 1$ at a certain step, and then since there was a path from $\ell_1$ to $t$, we assigned $t = 1$ and then we assigned $\ell_2 = 1$ (since it was not assigned a value) and then there was a path from $\ell_2$ to $\overline{t}$, we assign $t = 0$. However, in this case, there is a path from $t$ to $\overline{\ell}_2$, which means we already assigned a value to $\ell_2$ during the DFS visit of $\ell_1$. This shows that whenever our algorithm assigns a value to a variable, it must be consistent with prior assignments.

Finally, to conclude correctness, we need to show that the assignment obtained is consistent with all clauses. Suppose it is not consistent with some clause $(u, v)$. This means both $u$ and $v$ were assigned 0, which is same as saying that vertices corresponding to both $\overline{u}$ and $\overline{v}$

were visited during the execution of the algorithm. Without loss of generality, assume $\overline{u}$ was visited before $\overline{u}$. Since $(u, v)$ is a clause, the graph has an edge $(\overline{u}, v)$, and hence the algorithm assigns $v = 1$ contradicting our assumption. $\square$

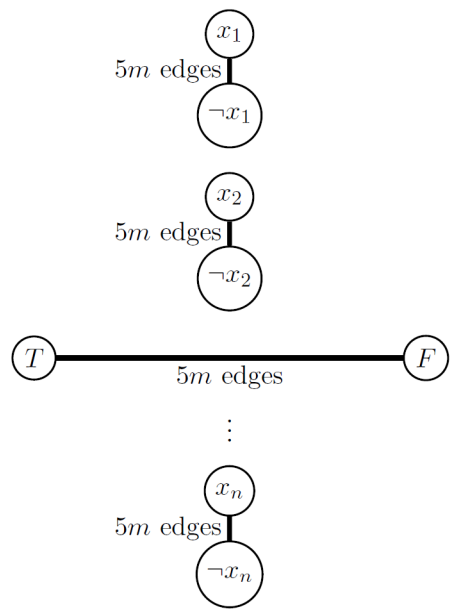


Figure 1: Starting graph for $G_\phi$, where $\phi$ has $n$ variables, $x_1, \ldots, x_n$.

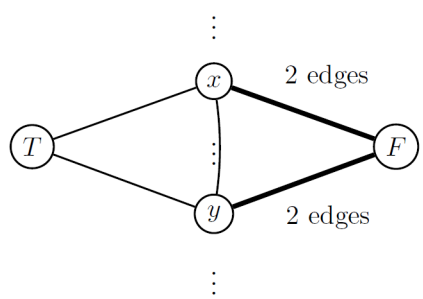


Figure 2: Edges to add for a clause of the form $(x \vee y)$.

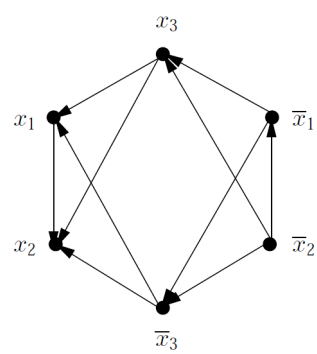


Figure 3: A graph capturing the implications associated with the following satisfiable instance of 2-SAT: $(x_3 \vee x_2) \wedge (x_3 \vee x_1) \wedge (\overline{x}_3 \vee x_2) \wedge (\overline{x}_1 \vee x_2) \wedge (\overline{x}_3 \vee x_1)$.